AnswerBot- AI-powered FAQ and customer support automation.

Introduction

→Project Title :AI FAQ Assistant API
→TEAM ID :SWTID-2026-1358
→TEAM LEADER :GOBIKA M B(gobikanadhika@gmail.com)
→TEAM MEMBER :GOKILASRI P (gokilasri550@gmail.com)
→TEAM MEMBER :GOMATHI SRI R (gomsgomathisri@gmail.com)
→TEAM MEMBER :GOPIKA A (angelingopika@gmail.com)

Description

The AI FAQ Assistant is a robust, secure, and intelligent REST API platform designed to streamline FAQ creation, user authentication, and AI-powered content management. Built with Node.js and Express.js, the application utilizes MongoDB as its primary data store along with Mongoose ODM for elegant data modeling and schema validation.

Security is central to the architecture, implementing JWT (JSON Web Tokens) for stateless user sessions and route guarding alongside bcrypt for strong password encryption. The backend handles intensive operations like automated FAQ generation, semantic search queries, and dynamic context parsing. It also features robust centralized error handling and response status sanitization to prevent unhandled database leakage and ensure operational durability.

Scenario-Based Case Study

Background

Priya is a customer support operations manager who handles massive backlogs of repetitive user inquiries but struggles to organize entries, classify topics, write precise summaries, and track client question trends efficiently.

Problem

- Difficulty generating and maintaining up-to-date documentation workflows.

- Limited visibility into semantic search match keywords and rapid query resolutions.
- Limited visibility into semantic search match keywords and rapid query resolutions.
- Time-consuming manual processes for drafting complex support answers.
- Lack of unified database permission layers protecting private documentation from public adjustments.

Solution

The AI FAQ Assistant backend delivers a centralized, secure API wrapper that decouples application business logic from frontend delivery systems. It introduces internal verification matrices controlling specialized services for content creation, category keywords, database persistence, and automated text inference modules.

Usage

- Authenticated Creator Usage: Registers profiles, requests instant AI assistance, drafts official FAQ schemas, and updates or removes self-authored records.
- Public/Consumer Usage: Queries general list routes to consume published articles and issues localized semantic queries via parameters.

Outcome

By offloading workflow controls to a robust backend, operational delays are minimized. Data structural integrity is strictly maintained through automated schema validation, security tokens guarantee that only original authors modify documents, and high-performance indexing facilitates real-time data discoverability.

1. Software Requirements

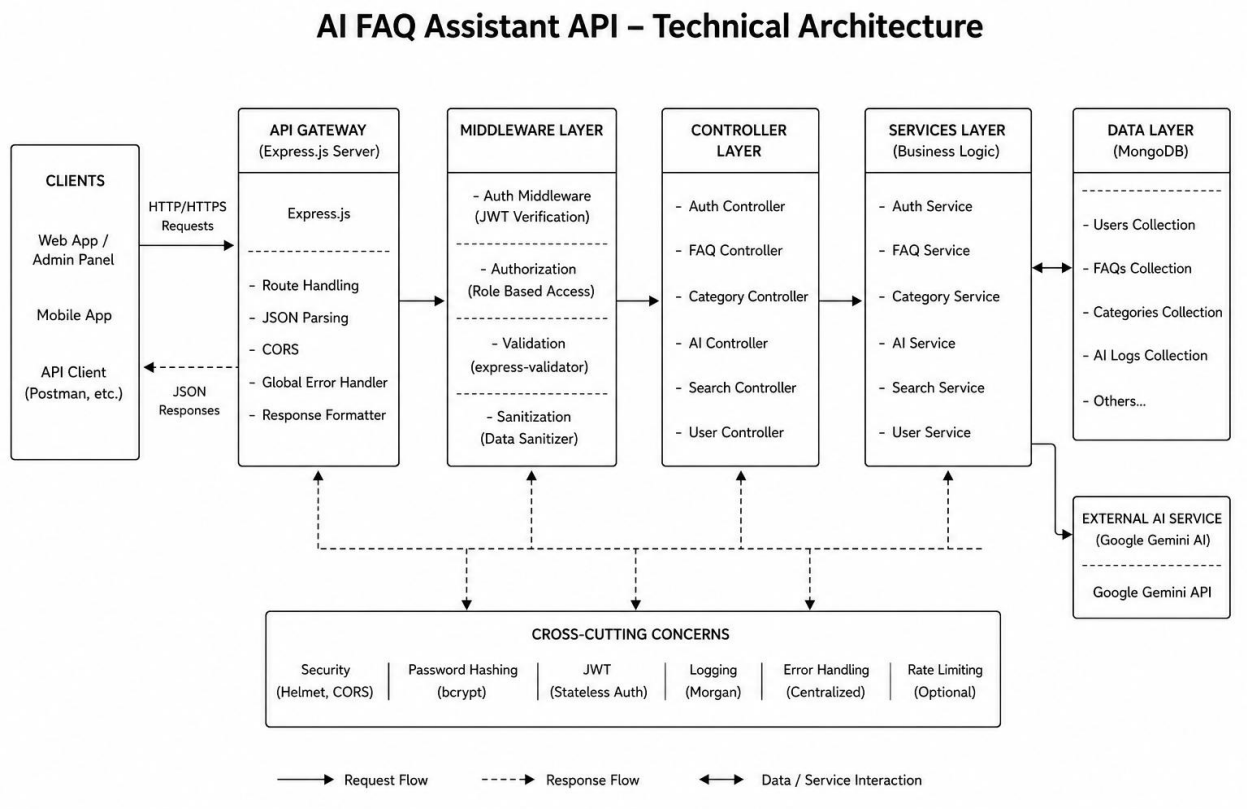
- Operating System: Windows 10/11, macOS, or Linux (supports cross-platform operations).
- Node.js (v18 or above): Runtime ecosystem running server logic and routing infrastructure.
- npm (v9 or above): Node package manager to control operational dependencies.
- Express.js (v4/v5 Ecosystem): Lightweight routing web framework to construct backend RESTful entry points.

- MongoDB / Mongoose ODM: Document NoSQL engine storing application users and curated data entities.
- Postman / ThunderClient: API verification toolkit to assert schema validation responses across administrative route shields.
- Code Editor: Visual Studio Code or similar IDE environment.

2. Hardware Requirements

- Processor: Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or equivalent.
- RAM: 8 GB minimum (16 GB recommended for concurrent instances of MongoDB processing layers).
- Storage: 1 GB of available disk workspace.

Epic-1: Project Architecture

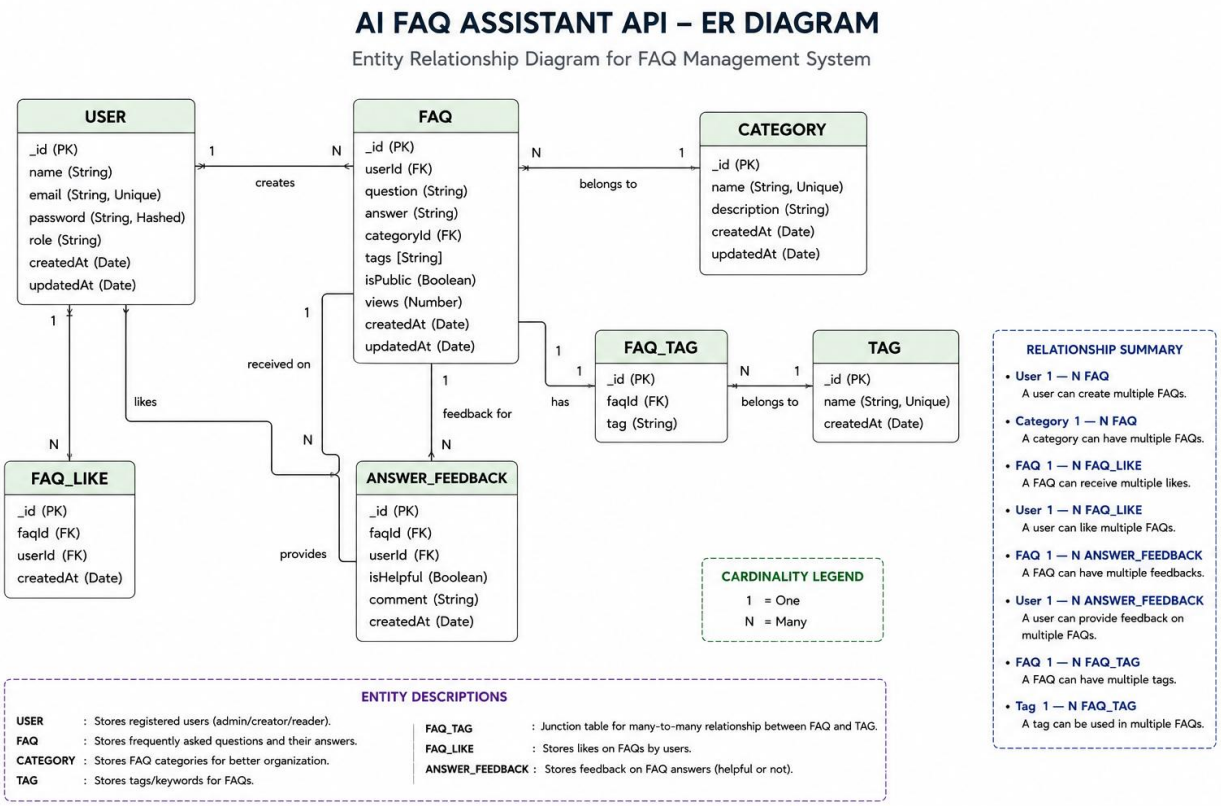


The AI FAQ Assistant application exposes structured endpoints handled by a layered server architecture model. Incoming requests undergo filter processing before interacting with data services.

Components Definition

- Interface: API tools like Postman query distinct route paths to extract JSON entities.
- Express Server Gateway: Monitors network ports, maps explicit HTTP methods (GET, POST, PUT, DELETE), parses incoming JSON payloads, and handles CORS headers.
- Authentication Middleware: Extracts tokens from headers, validates user context signatures, and evaluates authorization thresholds for guarded resource paths.
- Granular Services (Logic Modules): Enforces isolated domain rules for user session life cycles and operational query modifications.
- Database Interface Layer: Mongoose translates operational application logic into performant native MongoDB queries.
- AI Flow (Service Integration): Connected to external AI models via Google Gemini SDK integrations, letting the controller layer offload complex question-answering tasks and automated topic generation workflows.

Entity-Relationship (ER) Diagram Description



The backend database consists of multiple MongoDB collections connected through logical references to maintain user authentication, FAQ management, AI-generated content, and category organization.

---

1. User Entity

- `_id` (Primary Key - ObjectId)
- `name` (String, Required)
- `email` (String, Required, Unique)
- `password` (String, Hashed, Required)
- `role` (String, Default: "user")
- `createdAt`
- `updatedAt`

---

2. FAQ Entity

- `_id` (Primary Key - ObjectId)
- `question` (String, Required)
- `answer` (String, Required)
- `category` (String, Required)
- `createdBy` (Foreign Key referencing User Collection)
- `createdAt`
- `updatedAt`

---

3. AI Generation Entity (Logical)

(Generated dynamically using Google Gemini AI before persistence.)

- `topic` (String)
- `generatedQuestion` (String)

- `generatedAnswer` (String)
- `generatedCategory` (String)
- `generatedAt`

---

4. Category Entity (Logical)

- `categoryName` (String)
- `description` (String)
- `totalFAQs` (Number)

Key Structural Mappings

User → FAQ (One-to-Many):
A registered user can create and manage multiple FAQ records.

Category → FAQ (One-to-Many):
Each category can contain multiple FAQ documents.

User → AI Generated FAQ (One-to-Many):
A user can generate multiple AI-assisted FAQs, which may later be saved into the FAQ collection.

FAQ → Search Results (Logical Relationship):
Search queries retrieve matching FAQ documents using keyword and semantic matching.

Key Features

- **JWT Authentication & Authorization:** Secure token-based authentication protects private API endpoints.

- **AI FAQ Generation:** Google Gemini AI automatically generates questions, answers, and topic suggestions.
- **Semantic FAQ Search:** Intelligent keyword-based search enables faster information retrieval.
- **Complete CRUD Operations:** Create, Read, Update, and Delete FAQ records.
- **Category Management:** Organizes FAQs into structured categories for easier navigation.
- **Password Encryption:** User passwords are securely hashed using bcrypt before storage.
- **Centralized Error Handling:** Standardized API error responses improve reliability and debugging.
- **Schema Validation:** Mongoose validation ensures data integrity before database persistence.
- **RESTful API Architecture:** Modular Express.js routes provide scalable backend services.
- **MongoDB Integration:** Efficient NoSQL document storage using Mongoose ODM.

Roles and Responsibilities

Admin

- Manage all registered users.
- View, update, and delete any FAQ.
- Monitor AI-generated content.
- Manage FAQ categories.
- Access complete system functionality.

Content Creator

- Create new FAQs.
- Edit personal FAQ records.
- Delete owned FAQs.
- Generate FAQs using AI.
- Organize FAQs under categories.

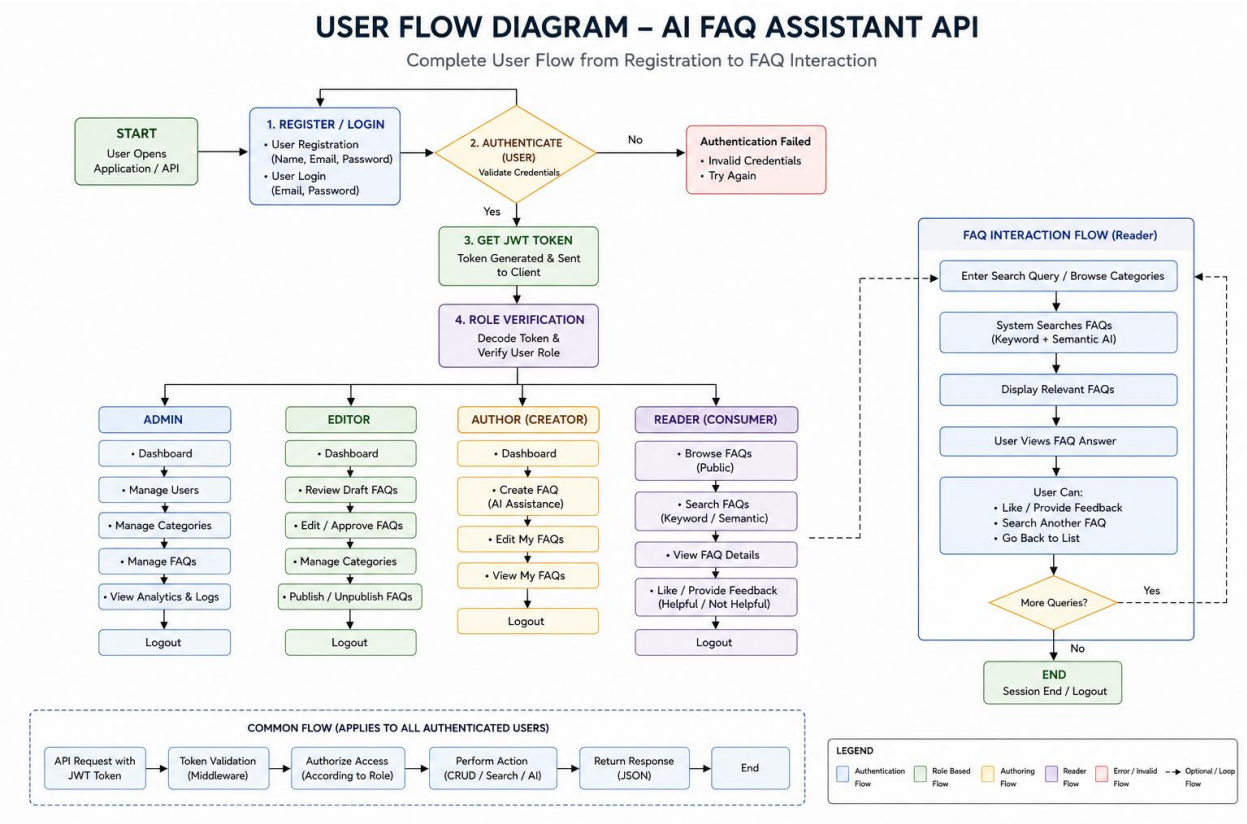
Authenticated User

- View public FAQs.
- Search FAQs using keywords.
- Generate AI-assisted answers.
- Access personal profile information.

Public User

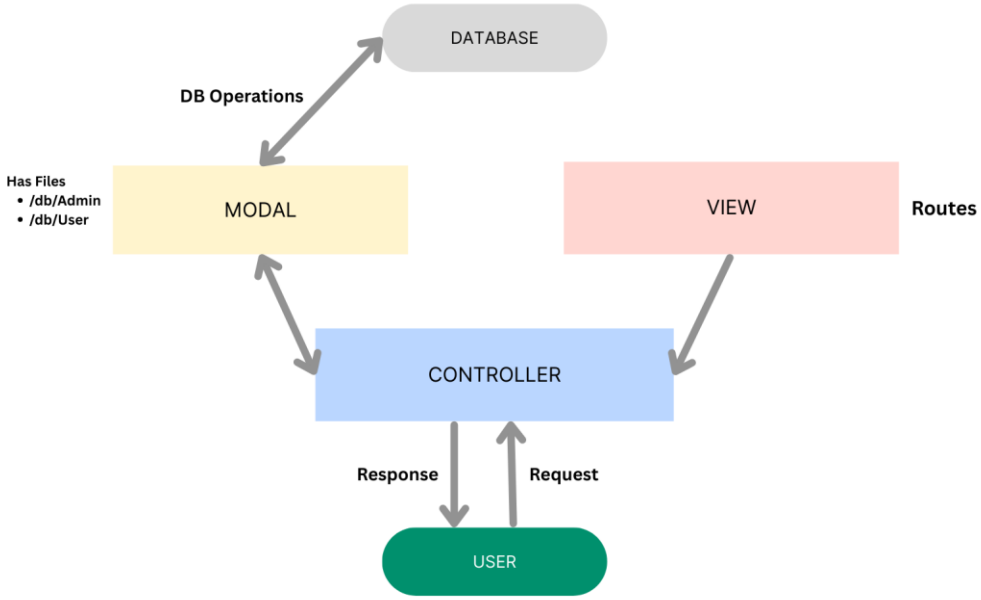
- View published FAQs.
- Search FAQs.
- Cannot create, edit, or delete FAQ records.
- No access to protected API endpoints.

User Flow



MVC Architecture Pattern

The system architecture structures its source configuration layers around the Model-View-Controller design pattern to completely decouple components:



The system architecture structures its source configuration layers around the Model-View-Controller design pattern to completely decouple components:

Model Layer (Data Blueprint)

Defines structural blueprints via strongly enforced Mongoose schemas mapping explicit fields into raw collection items inside MongoDB. It sets field rules and defaults before validation persistence occurs.

Controller Layer (Execution Matrix)

The intermediary brain of the ecosystem. It captures request vectors from execution routes, checks structural request constraints, delegates processing workflows directly to models or services, and packages resultant output into standard JSON return arrays.

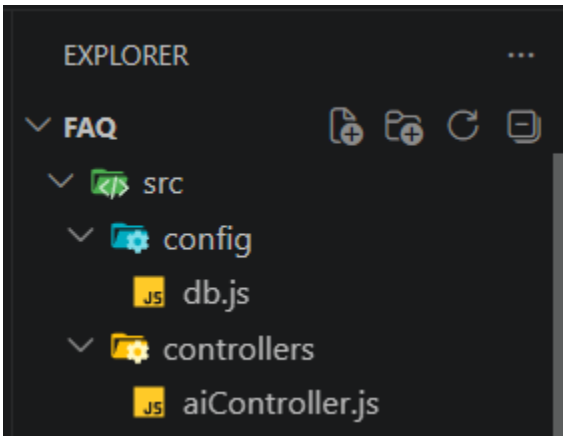
View Layer (Routing Network Interface)

In this pure headless backend service context, standard user-interface template views do not exist. Instead, the interface layer lives as an API Routing Layer. It links network requests on specified paths directly to their associated controller handler logic maps.

Epic 2: Project Setup and Configuration

Creating Project Folder

1. Create a new folder with your <project name>.
2. Inside that folder, create a new subfolder named src.
3. Now open that folder in VS Code.



Server Setup (npm init)

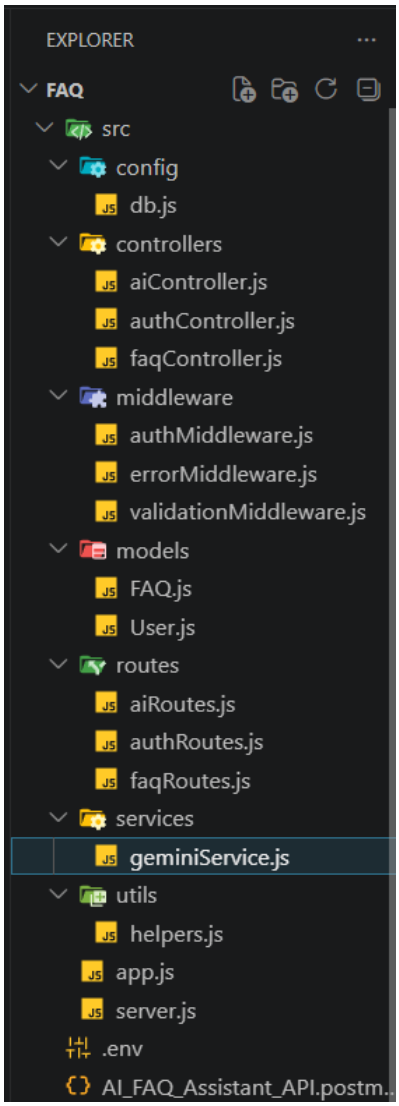
Open your root folder terminal in VS Code and run:

Bash

```
npm init -y
npm install express mongoose dotenv cors jsonwebtoken
bcrypt @google/genai
```

File and Folder Structure Matrix

Plaintext



Epic 3: Backend Development

Core System Script Implementations

1. Gateway Server Node (src/server.js)

This main entrance mechanism aggregates environment variables, triggers database listeners, and binds network listeners to active environment ports:

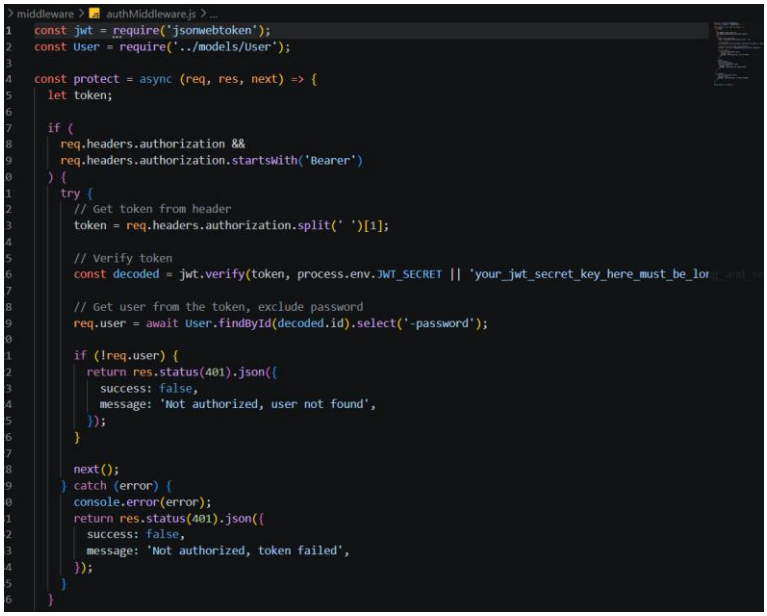
```
c > server.js > ...
1   const app = require('./app');
2   const connectDB = async () => {
3     try {
4       const dbConnect = require('./config/db');
5       await dbConnect();
6     } catch (error) {
7       console.error('Failed to connect to the database', error);
8     }
9   };
10
11  const startServer = async () => {
12    // Connect to database
13    await connectDB();
14
15    const PORT = process.env.PORT || 5000;
16
17    const server = app.listen(PORT, () => {
18      console.log(`Server running in ${process.env.NODE_ENV || 'development'} mode on port ${PORT}`);
19    });
20
21    // Handle unhandled promise rejections
22    process.on('unhandledRejection', (err, promise) => {
23      console.error(`Unhandled Rejection: ${err.message}`);
24      // Close server & exit process
25      server.close(() => process.exit(1));
26    });
27  };
28
29  startServer();
30
```

2. Gateway Express Setup (src/app.js)

```
.env   geminiService.js   app.js   AI_FAQ_Assistant_API.postman_collection.json   BLACKB
src > app.js > ...
1   const express = require('express');
2   const cors = require('cors');
3   const dotenv = require('dotenv');
4   const { errorHandler } = require('./middleware/errorMiddleware');
5
6   // Load environment variables
7   dotenv.config();
8
9   const app = express();
10
11  // Enable CORS
12  app.use(cors());
13
14  // Body parser middlewares
15  app.use(express.json());
16  app.use(express.urlencoded({ extended: false }));
17
18  // Basic status check route
19  app.get('/', (req, res) => {
20    res.json({
21      success: true,
22      message: 'AI FAQ Assistant API is running.',
23      version: '1.0.0',
24    });
25  });
26
27  // Register API Routes
28  app.use('/api/auth', require('./routes/authRoutes'));
29  app.use('/api/faqs', require('./routes/faqRoutes'));
30  app.use('/api/ai', require('./routes/aiRoutes'));
31
32  // Handle Socket.IO polling requests gracefully (if client attempts WebSocket connection)
33  app.use('/socket.io', (req, res) => {
34    res.status(404).json({ error: 'Socket.IO not enabled' });
35  });
36
```

3. Security Access Middleware (src/middleware/authMiddleware.js)

Intercepts incoming target queries, extracts transmission header authorization records, and validates token identity context claims:



```
middleware > authMiddleware.js > ...
const jwt = require('jsonwebtoken');
const User = require('../models/User');

const protect = async (req, res, next) => {
  let token;

  if (
    req.headers.authorization &&
    req.headers.authorization.startsWith('Bearer')
  ) {
    try {
      // Get token from header
      token = req.headers.authorization.split(' ')[1];

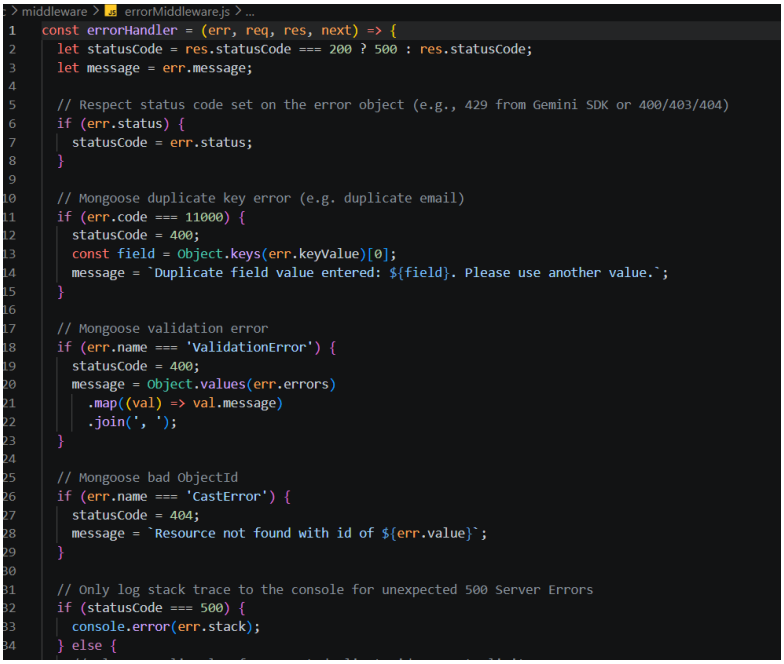
      // Verify token
      const decoded = jwt.verify(token, process.env.JWT_SECRET || 'your_jwt_secret_key_here_must_be_lon

      // Get user from the token, exclude password
      req.user = await User.findById(decoded.id).select('-password');

      if (!req.user) {
        return res.status(401).json({
          success: false,
          message: 'Not authorized, user not found',
        });
      }

      next();
    } catch (error) {
      console.error(error);
      return res.status(401).json({
        success: false,
        message: 'Not authorized, token failed',
      });
    }
  }
```

4. Centralized Error Handler Middleware (src/middleware/errorMiddleware.js)



```
middleware > errorMiddleware.js > ...
const errorHandler = (err, req, res, next) => {
  let statusCode = res.statusCode === 200 ? 500 : res.statusCode;
  let message = err.message;

  // Respect status code set on the error object (e.g., 429 from Gemini SDK or 400/403/404)
  if (err.status) {
    statusCode = err.status;
  }

  // Mongoose duplicate key error (e.g. duplicate email)
  if (err.code === 11000) {
    statusCode = 400;
    const field = Object.keys(err.keyValue)[0];
    message = `Duplicate field value entered: ${field}. Please use another value.`;
  }

  // Mongoose validation error
  if (err.name === 'ValidationError') {
    statusCode = 400;
    message = Object.values(err.errors)
      .map((val) => val.message)
      .join(', ');
  }

  // Mongoose bad ObjectId
  if (err.name === 'CastError') {
    statusCode = 404;
    message = `Resource not found with id of ${err.value}`;
  }

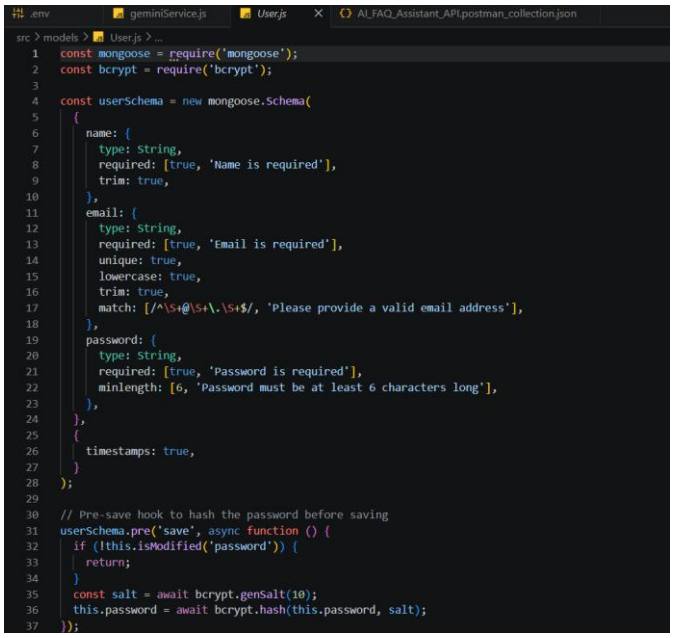
  // Only log stack trace to the console for unexpected 500 Server Errors
  if (statusCode === 500) {
    console.error(err.stack);
  } else {
```

Epic 4: Database Configuration

Mongoose Schema Definitions

1. Identity Resource Document Schema (src/models/User.js)

Enforces profile field criteria validations and hooks password hashing steps prior to user document persistence:



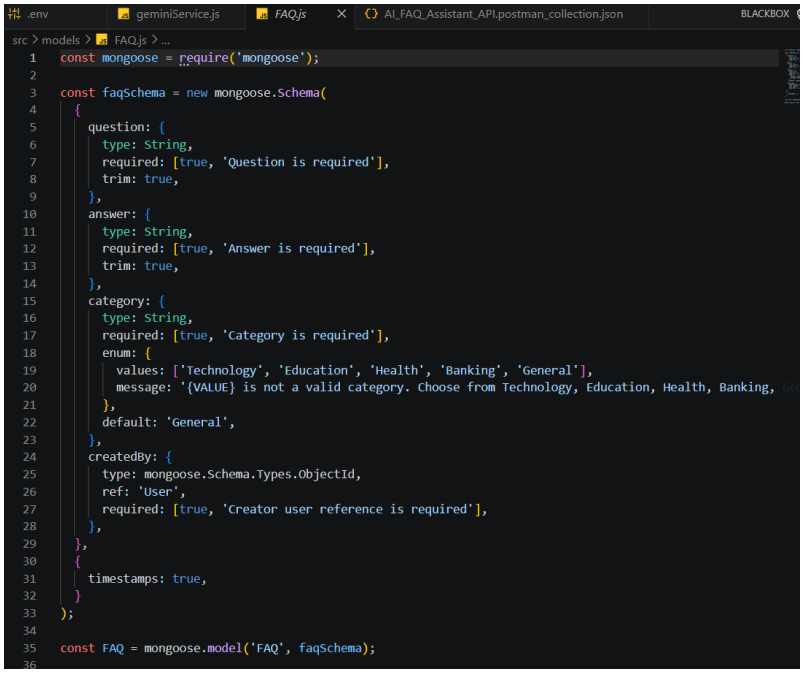
```
const mongoose = require('mongoose');
const bcrypt = require('bcrypt');

const userSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: [true, 'Name is required'],
      trim: true,
    },
    email: {
      type: String,
      required: [true, 'Email is required'],
      unique: true,
      lowercase: true,
      trim: true,
      match: [/^\S+@\S+\.\S+$/, 'Please provide a valid email address'],
    },
    password: {
      type: String,
      required: [true, 'Password is required'],
      minlength: [6, 'Password must be at least 6 characters long'],
    },
  },
  {
    timestamps: true,
  }
);

// Pre-save hook to hash the password before saving
userSchema.pre('save', async function () {
  if (!this.isModified('password')) {
    return;
  }
  const salt = await bcrypt.genSalt(10);
  this.password = await bcrypt.hash(this.password, salt);
});
```

2. FAQ Resource Document Schema (src/models/FAQ.js)

Models core properties and maps a distinct relational reference link back to the source creator document:



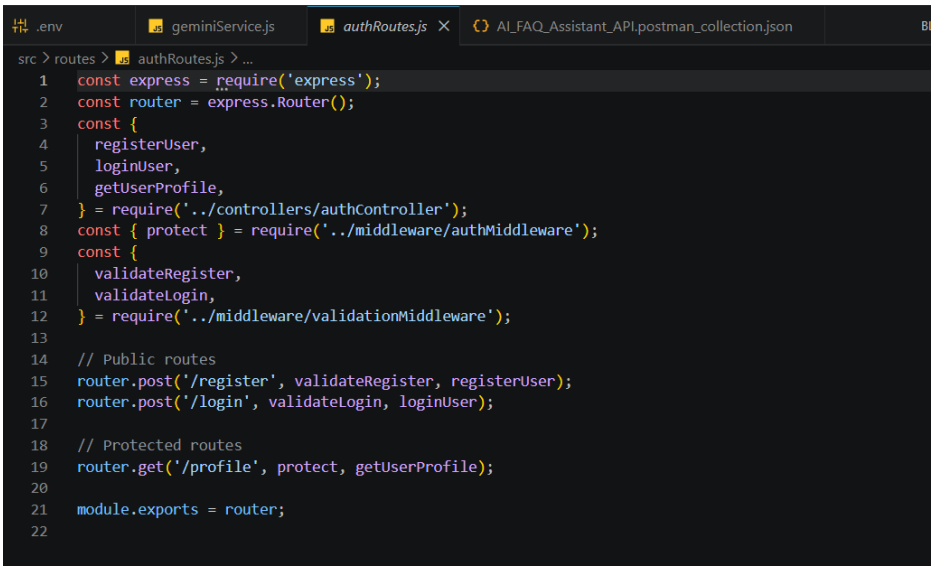
```
const mongoose = require('mongoose');

const faqSchema = new mongoose.Schema(
  {
    question: {
      type: String,
      required: [true, 'Question is required'],
      trim: true,
    },
    answer: {
      type: String,
      required: [true, 'Answer is required'],
      trim: true,
    },
    category: {
      type: String,
      required: [true, 'Category is required'],
      enum: {
        values: ['Technology', 'Education', 'Health', 'Banking', 'General'],
        message: '{VALUE} is not a valid category. Choose from Technology, Education, Health, Banking,
      },
      default: 'General',
    },
    createdBy: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User',
      required: [true, 'Creator user reference is required'],
    },
  },
  {
    timestamps: true,
  }
);

const FAQ = mongoose.model('FAQ', faqSchema);
```

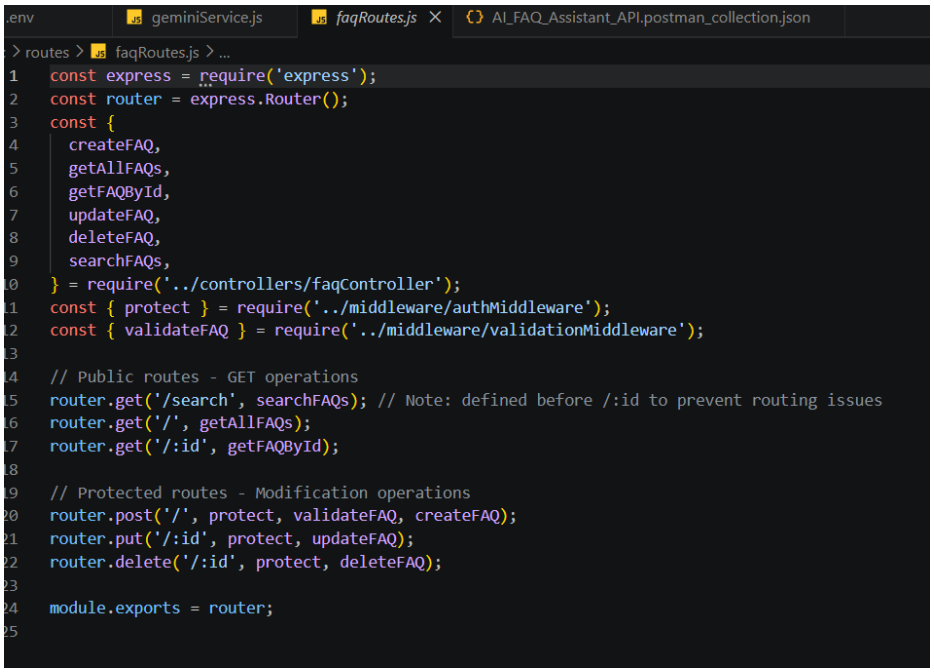
API Router Interface Endpoints

1. Public Authentication Operations (src/routes/authRoutes.js)

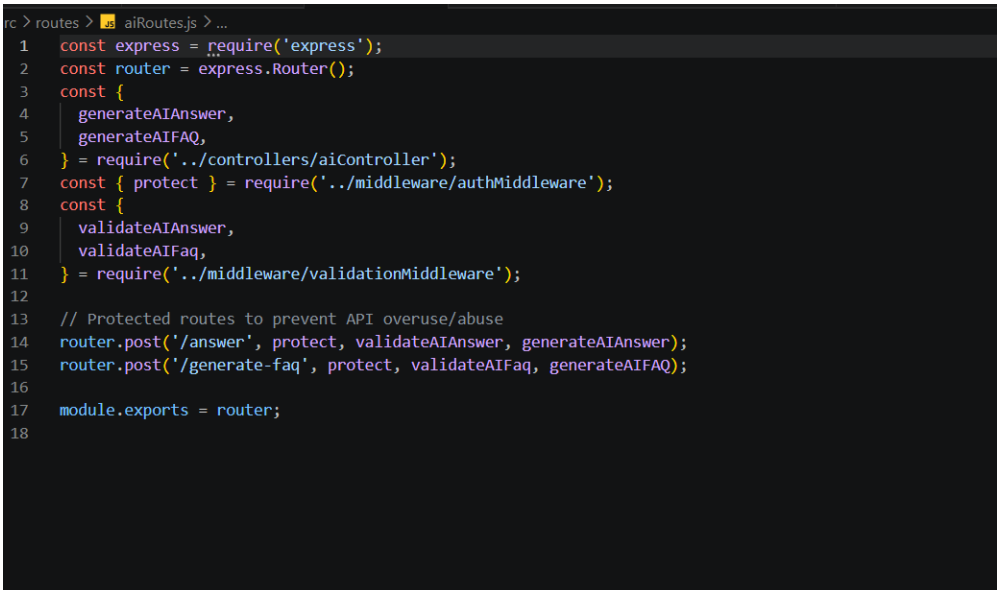


2. Protected FAQ Operations (src/routes/faqRoutes.js)

JavaScript



3. AI Automation Operations (src/routes/aiRoutes.js)



```
src > routes > aiRoutes.js > ...
1   const express = require('express');
2   const router = express.Router();
3   const {
4     generateAIAnswer,
5     generateAIFAQ,
6   } = require('../controllers/aiController');
7   const { protect } = require('../middleware/authMiddleware');
8   const {
9     validateAIAnswer,
10    validateAIFaq,
11  } = require('../middleware/validationMiddleware');
12
13  // Protected routes to prevent API overuse/abuse
14  router.post('/answer', protect, validateAIAnswer, generateAIAnswer);
15  router.post('/generate-faq', protect, validateAIFaq, generateAIFAQ);
16
17  module.exports = router;
18
```

Epic 5: Project Executions

Running the API Application

- Install Postman or ThunderClient (VS Code extension).
- Start the backend development server environment parameters:

Bash

```
npm run dev
```

API Testing Specification Demo

To verify operational structural integrity via execution endpoints, map endpoints using the configurations below:

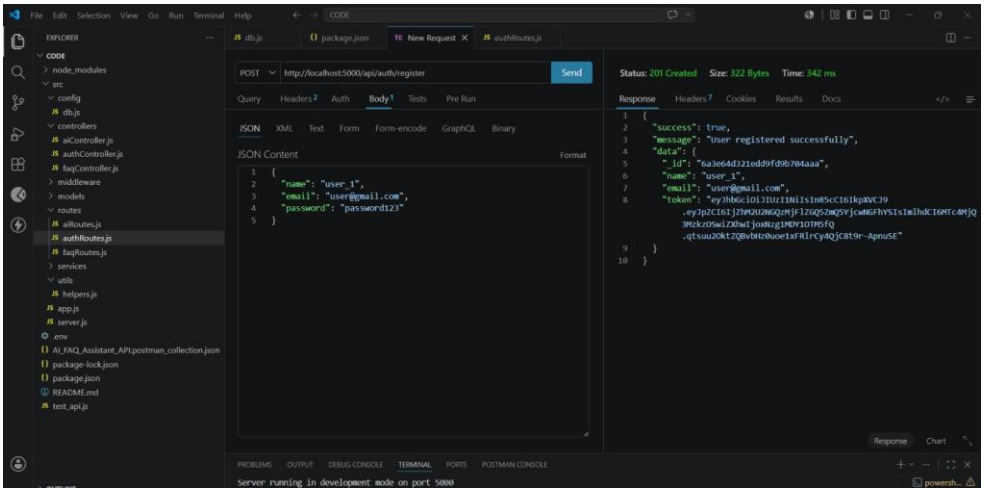
1. Identity Onboarding Path Registration

- Route: POST /api/auth/register
- Access: Public
- Payload Request Body JSON:

JSON

```
{
  "name": "Priya Sharma",
  "email": "priya@writeflow.com",
  "password": "securepassword123"
}
```

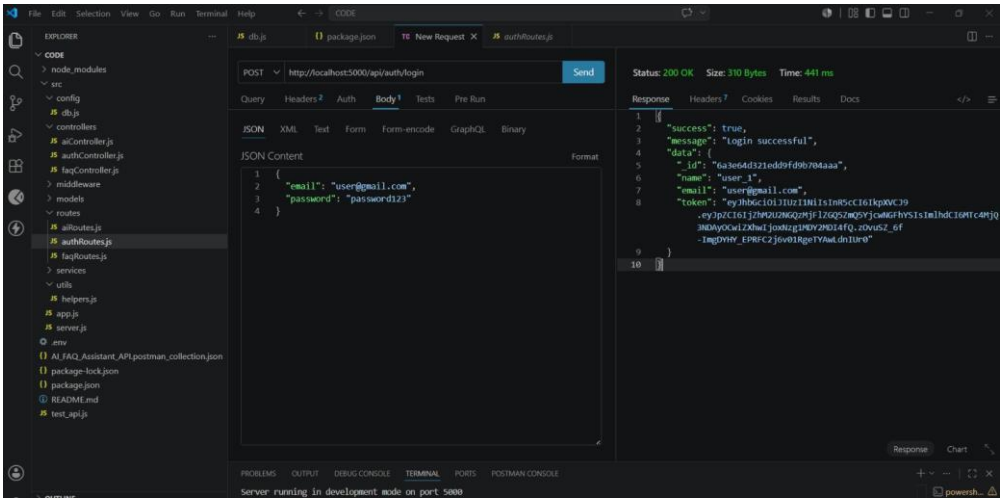
2. Profile Login Verification Quey



- Route: POST /api/auth/login
- Access: Public
- Payload Request Body JSON:

JSON

```
{
  "email": "priya@writeflow.com",
  "password": "securepassword123"
}
```

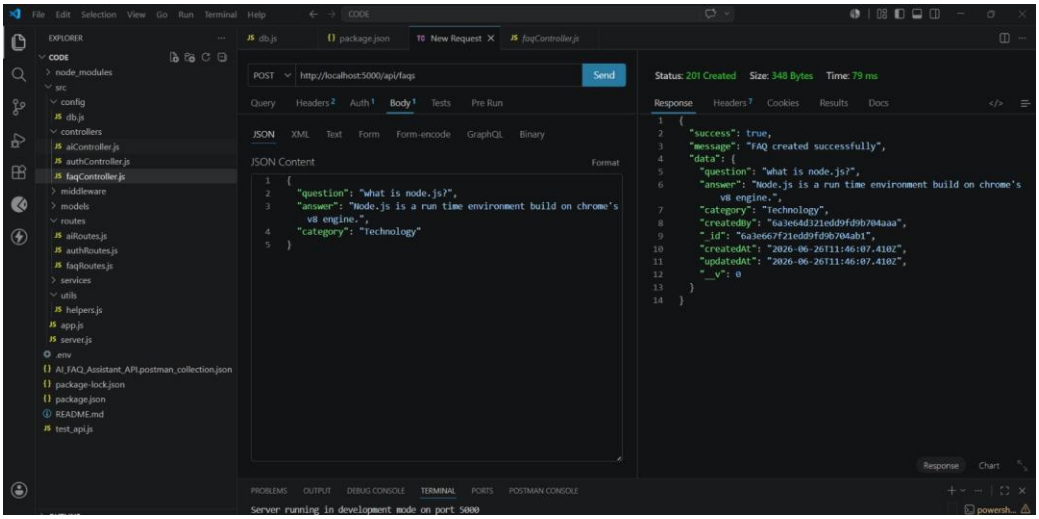


3. Resource Entity Creation (FAQ)

- Route: POST /api/faqs
- Access: Private (Bearer <JWT Token>)
- Payload Request Body JSON:

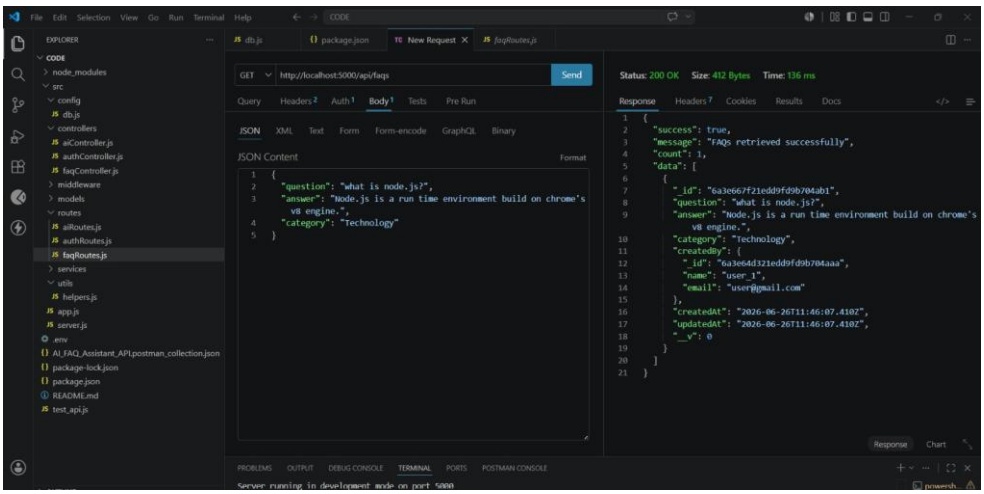
JSON

```
{
  "question": "How do I configure custom category tags?",
  "answer": "Navigate to settings panel, append keyword tags arrays, and save schema.",
  "category": "Configuration"
}
```



4. Keyword Resource Entity Fetching (Search)

- Route: GET /api/faqs/search?q=configure
- Access: Public



5. AI Intelligent Content Automation (Generate FAQ Pair)

- Route: POST /api/ai/generate-faq
- Access: Private (Bearer <JWT Token>)
- Payload Request Body JSON:

JSON

```
{
  "topic": "Mongoose schema indexing validation runtime workflow optimization"
}
```